

ห้ามมิให้นักศึกษาทำซ้ำ ดัดแปลง หรือใช้ประโยชน์จากงานอันมีลิขสิทธิ์ปรากฏอยู่ในระบบการศึกษาอิเล็กทรอนิกส์ (E-Learning) ของมหาวิทยาลัย ไม่ว่าจะทั้งหมดหรือบางส่วน โดยไม่ได้รับอนุญาตจากมหาวิทยาลัย นอกเหนือจากการศึกษาส่วนบุคคล ทั้งนี้ การทำซ้ำ ดัดแปลง หรือเผยแพร่ต่อสาธารณชนซึ่งงานอันมีลิขสิทธิ์ **จะมีโทษปรับตั้งแต่ 20,000 บาท ถึง 200,000 บาท และหากเป็นการกระทำเพื่อการค้า จะมีโทษจำคุกตั้งแต่ 6 เดือน ถึง 4 ปี หรือปรับตั้งแต่ 100,000 บาท ถึง 800,000 บาท หรือทั้งจำทั้งปรับ**

Unless allowed by Mahidol University, the User shall not copy, modify, or exploit in part or in whole of the copyrighted materials on the Platform, other than for your own individual study. The copy, modify or communication to public of the copyrighted materials shall be inflicted with a fine from 20,000 Baht up to 200,000 Baht. If the offence is committed with the commercial purpose, the offender shall be inflicted with imprisonment for a term from 6 months up to 4 years or a fine from 100,000 Baht up to 800,000 Baht or both imprisonment and fine.

Mahidol University
Faculty of Information
and Communication Technology



ITDS261 Introduction to Software Engineering

Lecture 11 – Version Control System

Topics

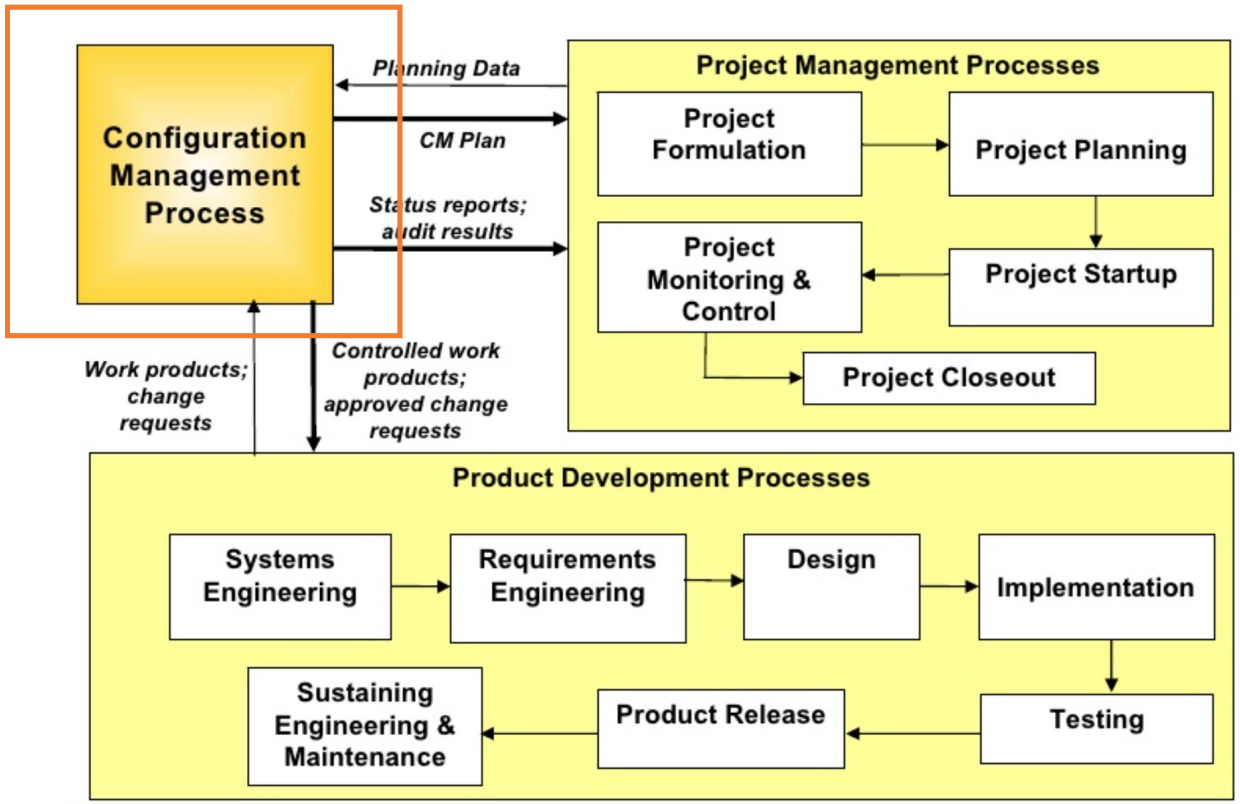
- Software configuration management
- Version control

Software Configuration Management (CM)

- Software Configuration Management encompasses the disciplines and techniques of initiating, evaluating and controlling change to software products during and after the software engineering process.

- Standards:
  IEEE 828: Software Configuration Management Plans
  IEEE 1042: Guide to Software Configuration Management

Configuration management process



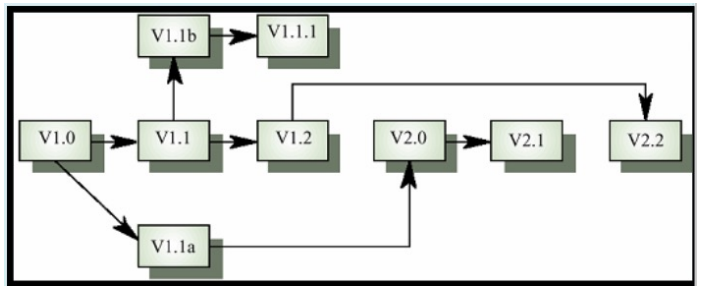
Elliando Dias, Software Configuration Management, 2008

Configuration item

- A piece of software, a document, a piece of hardware or work product which is subject to change is a configuration item
- Configuration items are ‘things’ we want to keep track of and control.
- Almost anything can be a software configuration item
- Not only program code segments but all type of documents according to development,
  - all type of code files
  - drivers for tests
  - Requirement, analysis, and design documents
  - user or developer manuals
  - system configurations (e.g. version of compiler used)
  - hardware configuration items (CPUs, bus speed frequencies)

Version identification

- Procedures for version identification should define an unambiguous way of identifying component versions
- Basic techniques for component identification:
  - 1) Version numbering:
    - Simple naming scheme uses a linear derivation
    - e.g. V1, V1.1, V1.2, V2.1, V2.2 etc.
    - Names are not meaningful.
  - 2) Attribute-based identification:
    - Attributes can be associated with a version with the combination of attributes identifying that version.
    - Examples of attributes are Date, Creator, Programming Language, Customer, Status.



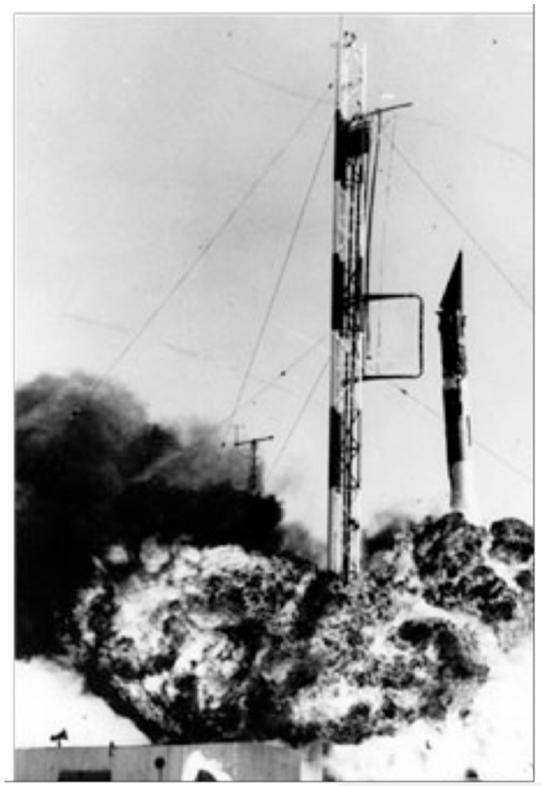


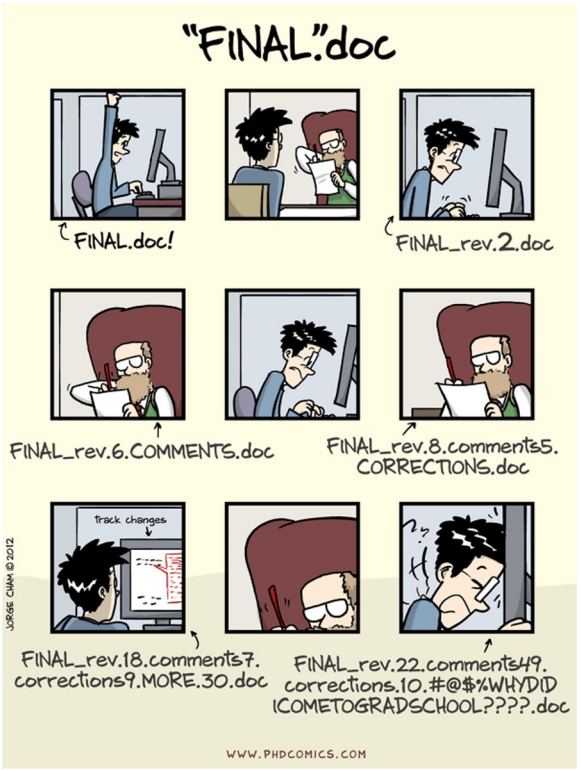
Mahidol University
Faculty of Information
and Communication Technology



Version Control

Losing track of your changes?





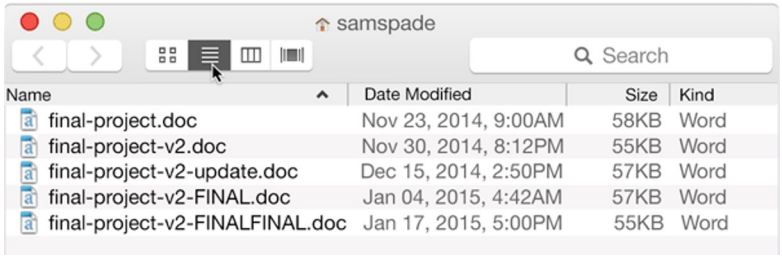


Photo: https://www.nasa.gov/multimedia/imagegallery/image_feature_926.html

Version Control System (VCS)

- It is a component in configuration management.
- Purposes:
  1. Restoring previous configurations
  2. Coordinating changes
  3. Identifying versions and components

Version Control Systems

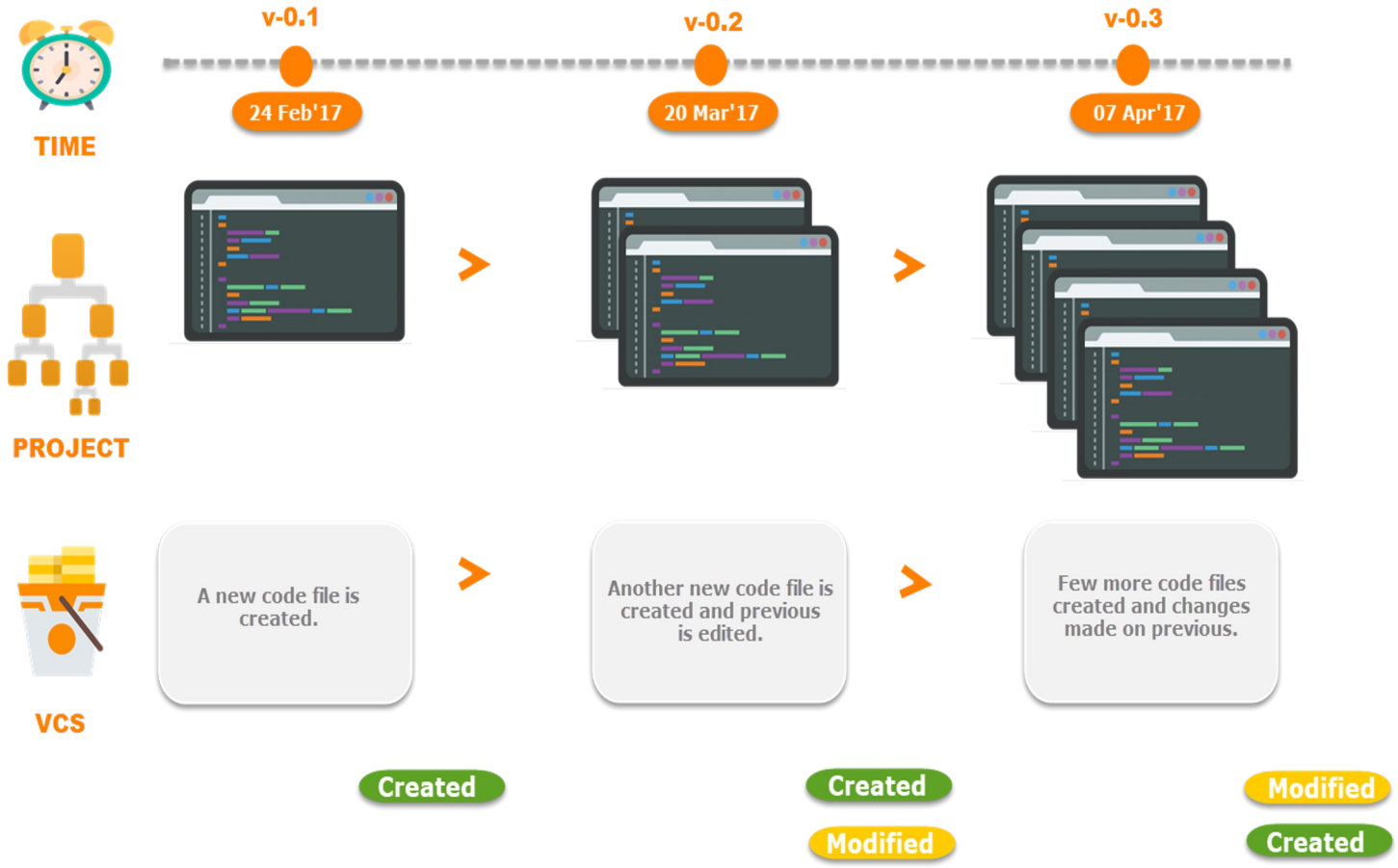
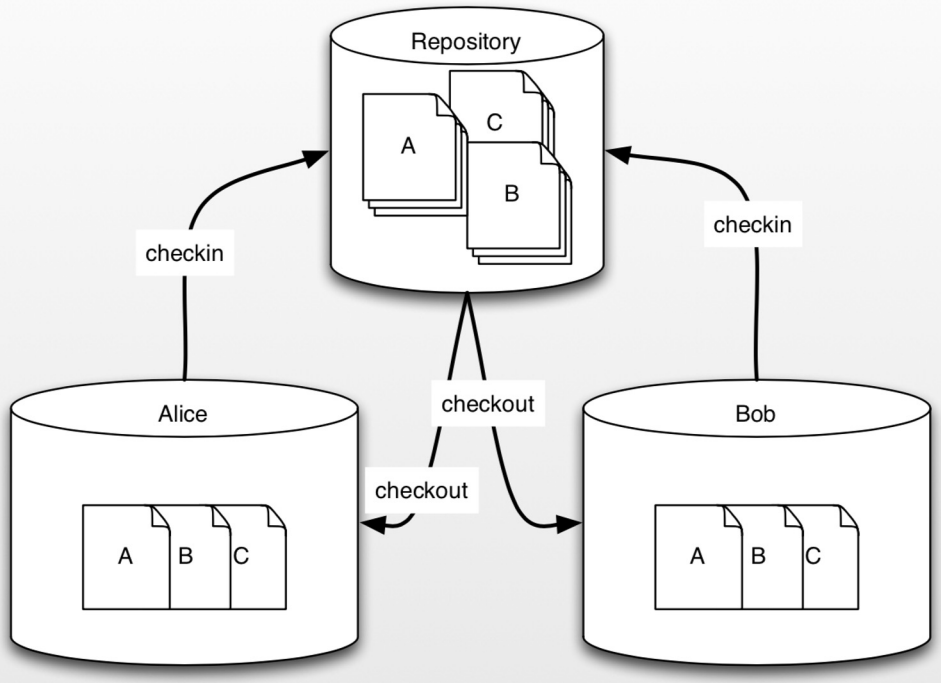
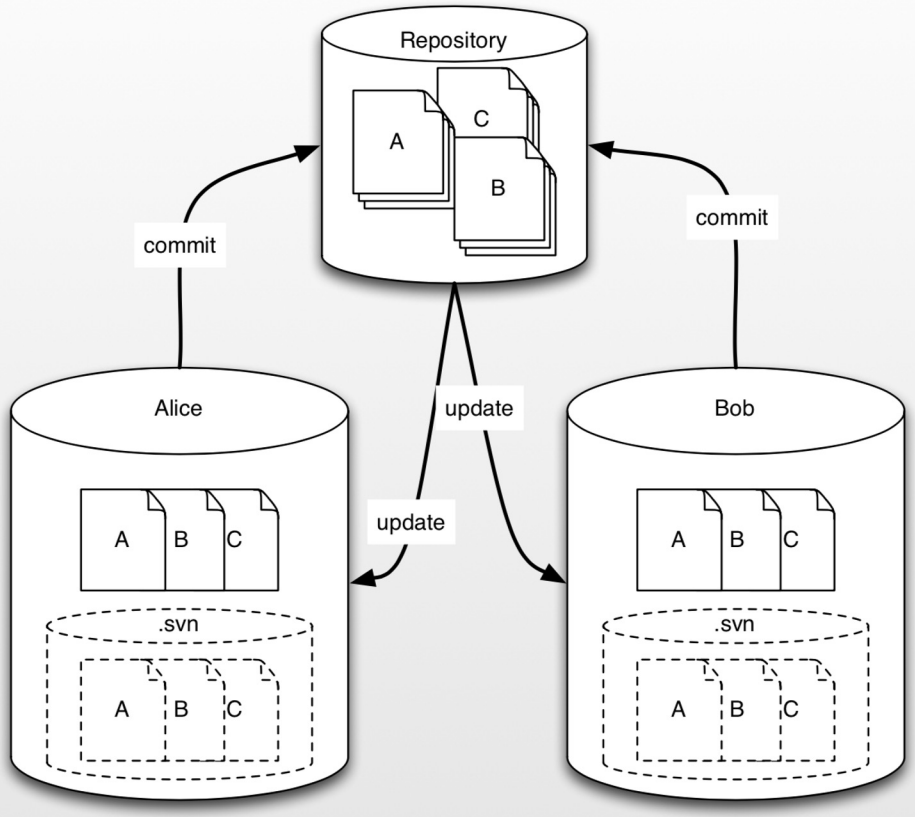


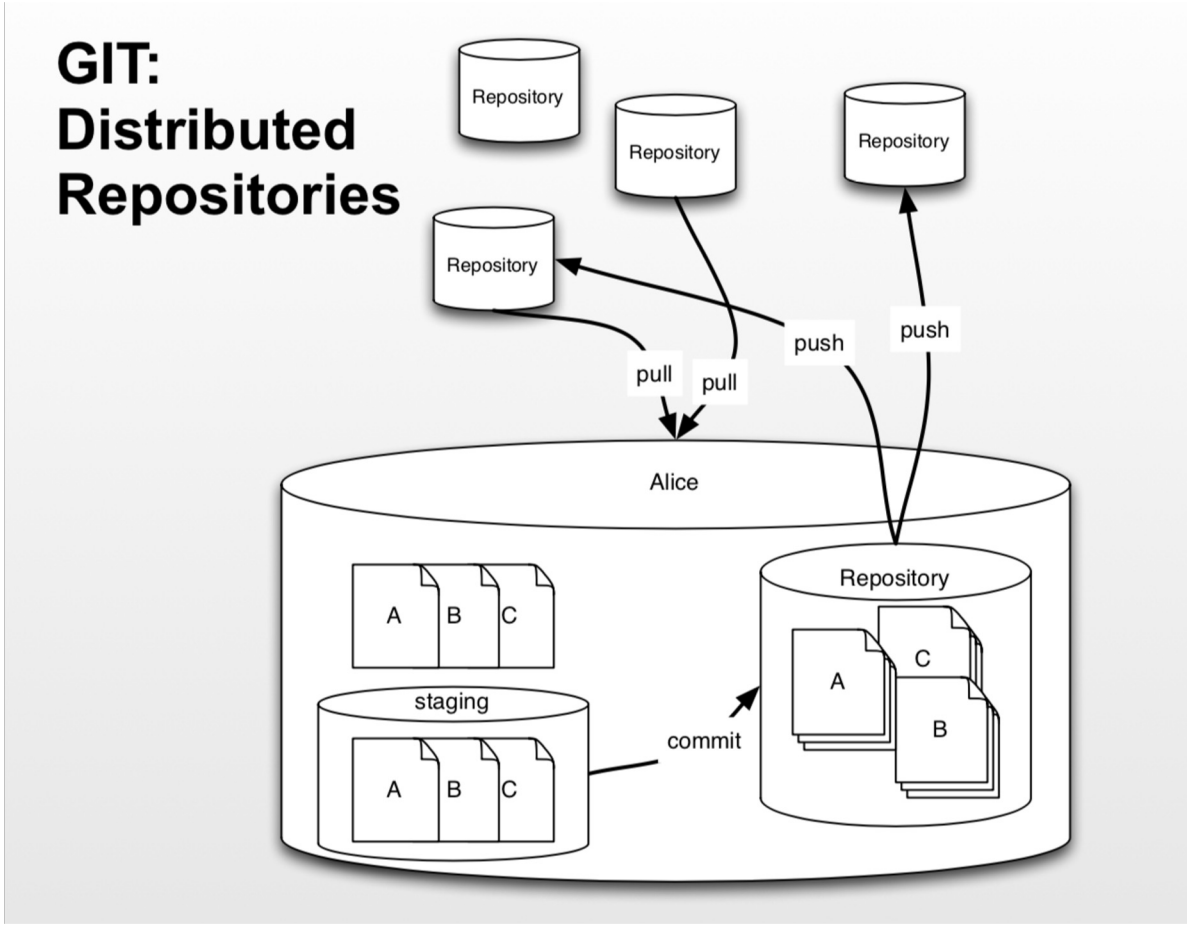
Image: https://www.toolsqa.com/git/version-control-system/

CVS


Repository
A
C
B
checkin
checkin
checkout
checkout
Alice
Bob
A
B
C
A
B
C

SUBVERSION


Repository
A
C
B
commit
commit
Alice
update
update
Bob
A
B
C
.svn
A
B
C
A
B
C
.svn
A
B
C


GIT:
Distributed
Repositories
Repository
Repository
Repository
Repository
pull
pull
push
push
Alice
A
B
C
staging
A
B
C
commit
Repository
A
C
B

Git is a free and open-source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.



- Easy to learn
- Tiny footprint with lightning-fast performance.
- Overshadow other SCM tools like Subversion and CVS
- Cheap local branching, convenient staging areas, and multiple workflows.

Reference: https://git-scm.com

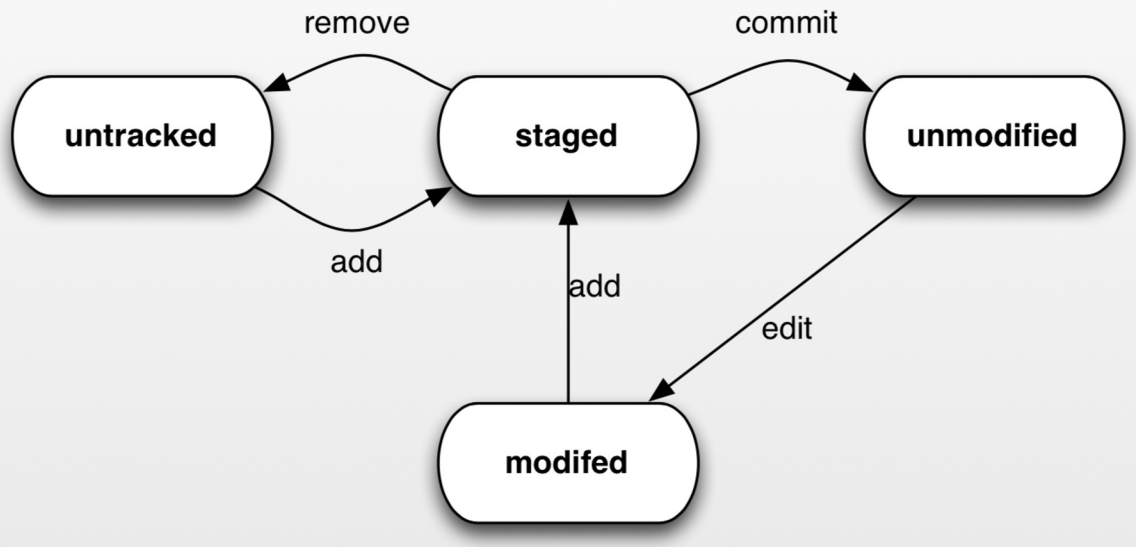
Distributed Version Control

- Almost everything is local
- Almost every operation is fast
- Almost every cloned repository is a backup
- Almost every operation works offline
- Almost all repositories have the full history

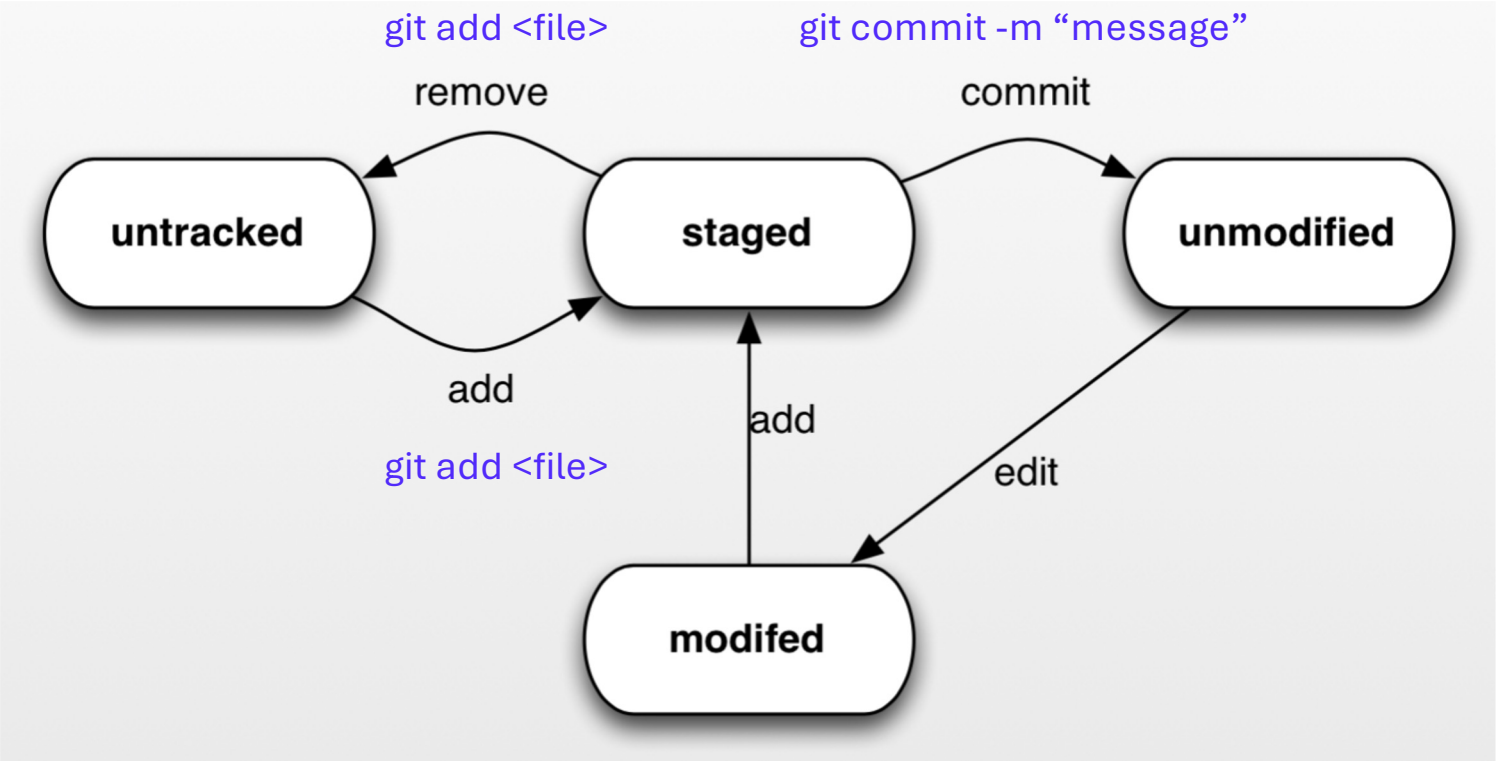
GIT's Special: The Staging Area

- Unlike other version control systems, GIT has a concept of the staging area (index)
- These are changes which will be committed.
- The staging area can be used to prepare for the commit.

File Status Lifecycle



Git Staging Operations



Add

- Adding selected changes from the working directory to the staging area.
- Specifies which changes will be included in the next commit.
- Changes can be reviewed before committing.

Commit

- Adding the staged changes to the local repository.
- Creates a new version (commit) of the project.
- A commit message describes the changes made.

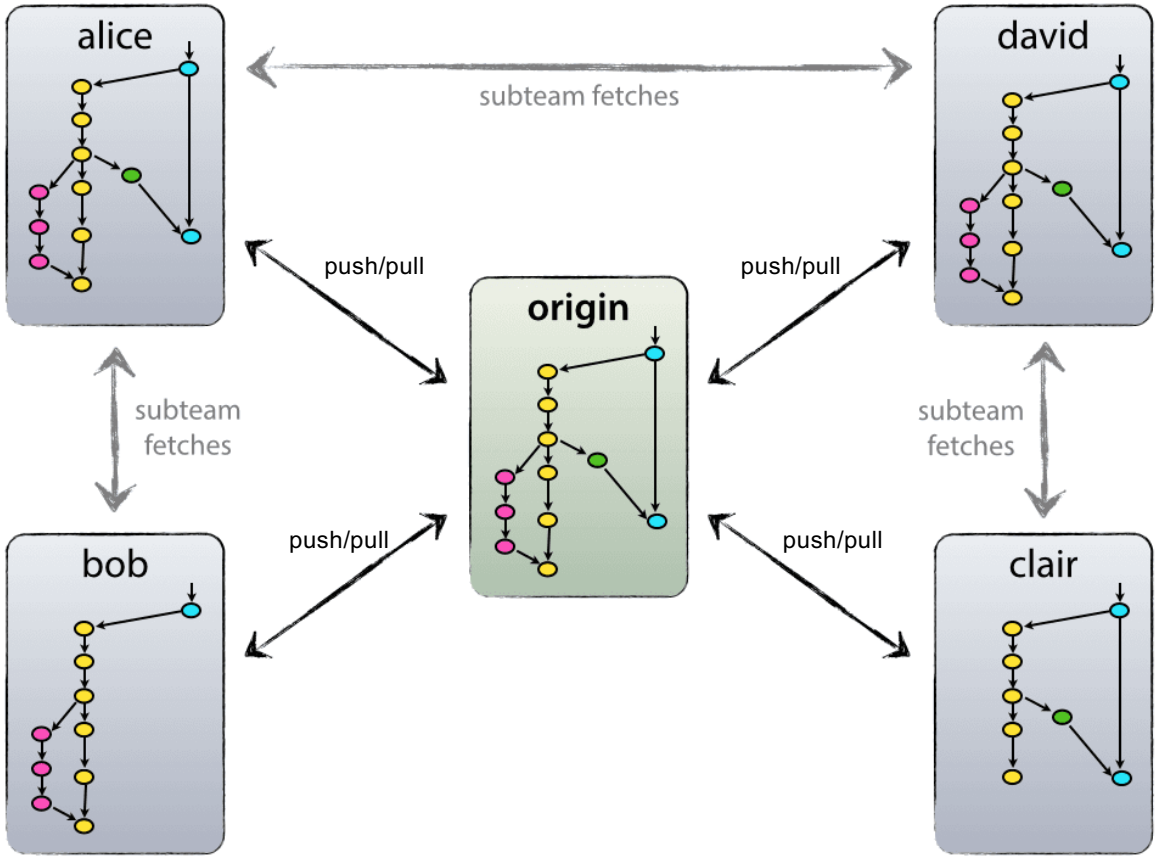
Pull

- Adding the changes from remote repositories to the local repository.
- Fetches the remote changes and merges them into the current branch.
- Merges are usually automatic.
- Conflicts will occur, but relatively easy to deal with.

Push

- Adding the local commits to the remote repository.
- Makes local changes available to other collaborators.
- Push may be rejected if the remote repository contains changes that are not in the local repository.
- Propagating outside push/pull:
  - GIT can create patches
  - GIT can integrate patches

Decentralized but Centralized



Source: https://nvie.com/posts/a-successful-git-branching-model/

What about Branches?

- Git has one default branch called the "master" branch.
- The master branch is deployable.
- It is your production code, ready to roll out into the world.
- The master branch is meant to be stable.
- It is the social contract of open-source software to never, ever push anything to master that is not tested, or that breaks the build.
- The entire reason GitHub works is that it is always safe to work from the master.

Reference: https://thenewstack.io/dont-mess-with-the-master-working-with-branches-in-git-and-github/

When to Create a New Branch?

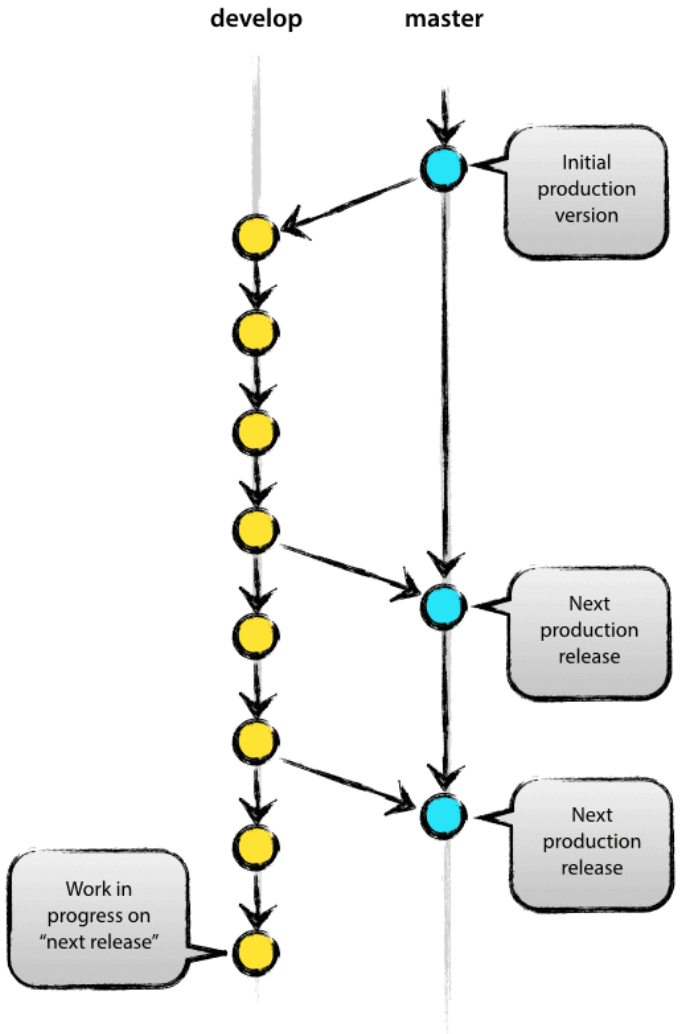
- Adding a new feature
- Working on a new release of the software
- Experimenting with some new changes
- Fixing a bug

Branching and Merging

- Branching and merging is simple in git.
- Branches of often temporary.
- Branching is encouraged.
- Merging is mainly automatic.
- However, branching can cause problems due to conflicts at merge.
- Resolving conflicts without tool support is painful.
- One should adapt a defined workflow.

Git Branching Model

- The main branches alwasy include master and develop
- master to be the main branch where the source code always reflects a *production-ready* state.
- develop to be the main branch where the source code always reflects a state with the latest delivered development changes for the next release.



Source: https://nvie.com/posts/a-successful-git-branching-model/

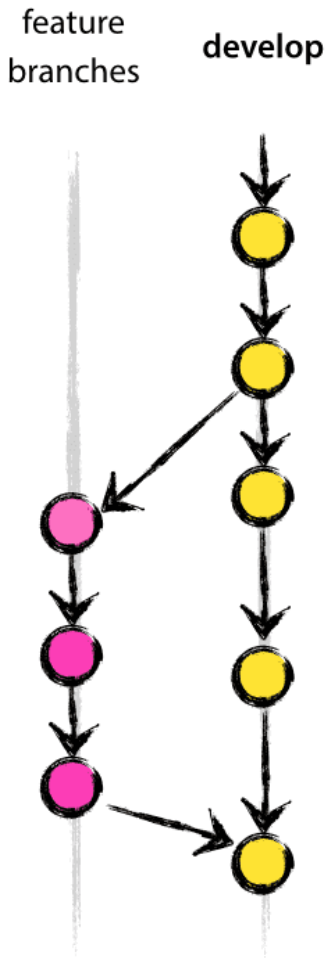
Git Branching Model (cont.)

- Supporting branches
  - Feature branches
  - Release branches
  - Hotfix branches

Source: https://nvie.com/posts/a-successful-git-branching-model/

Feature Branches

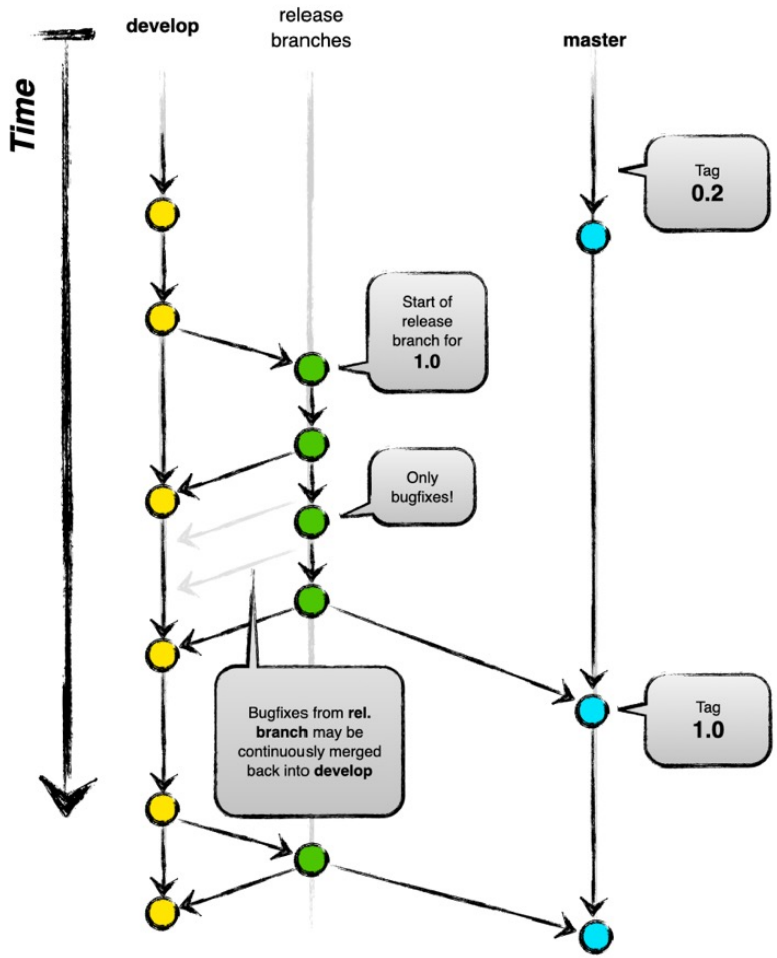
- Feature branches (or sometimes called topic branches) are used to develop new features for the upcoming or a distant future release.



Source: https://nvie.com/posts/a-successful-git-branching-model/

Release Branches

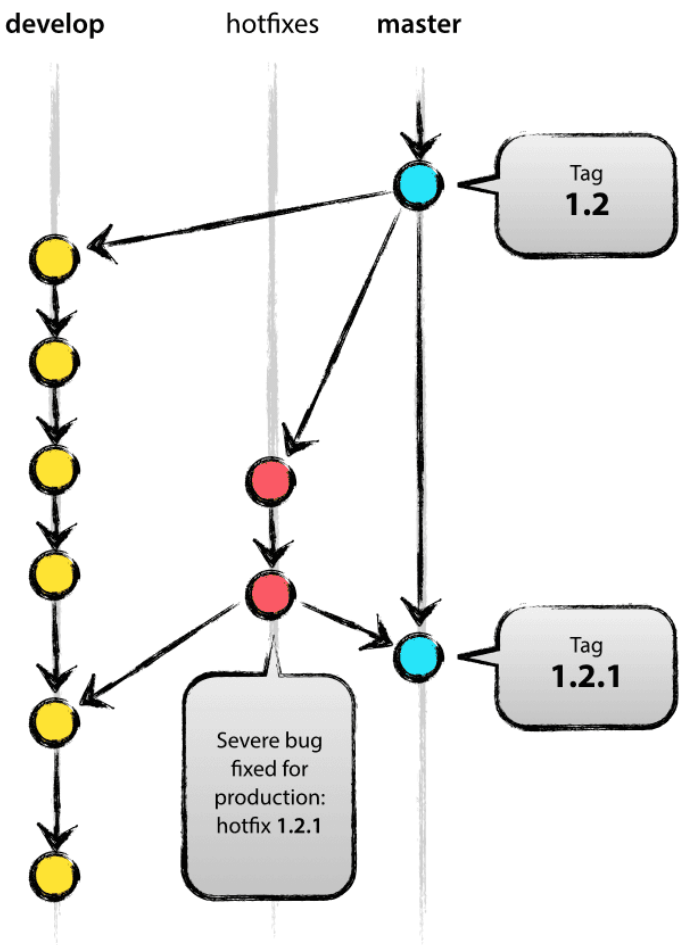
- Release branches support preparation of a new production release.
- By doing all of this work on a release branch, the develop branch is cleared to receive features for the next big release.



Source: https://nvie.com/posts/a-successful-git-branching-model/

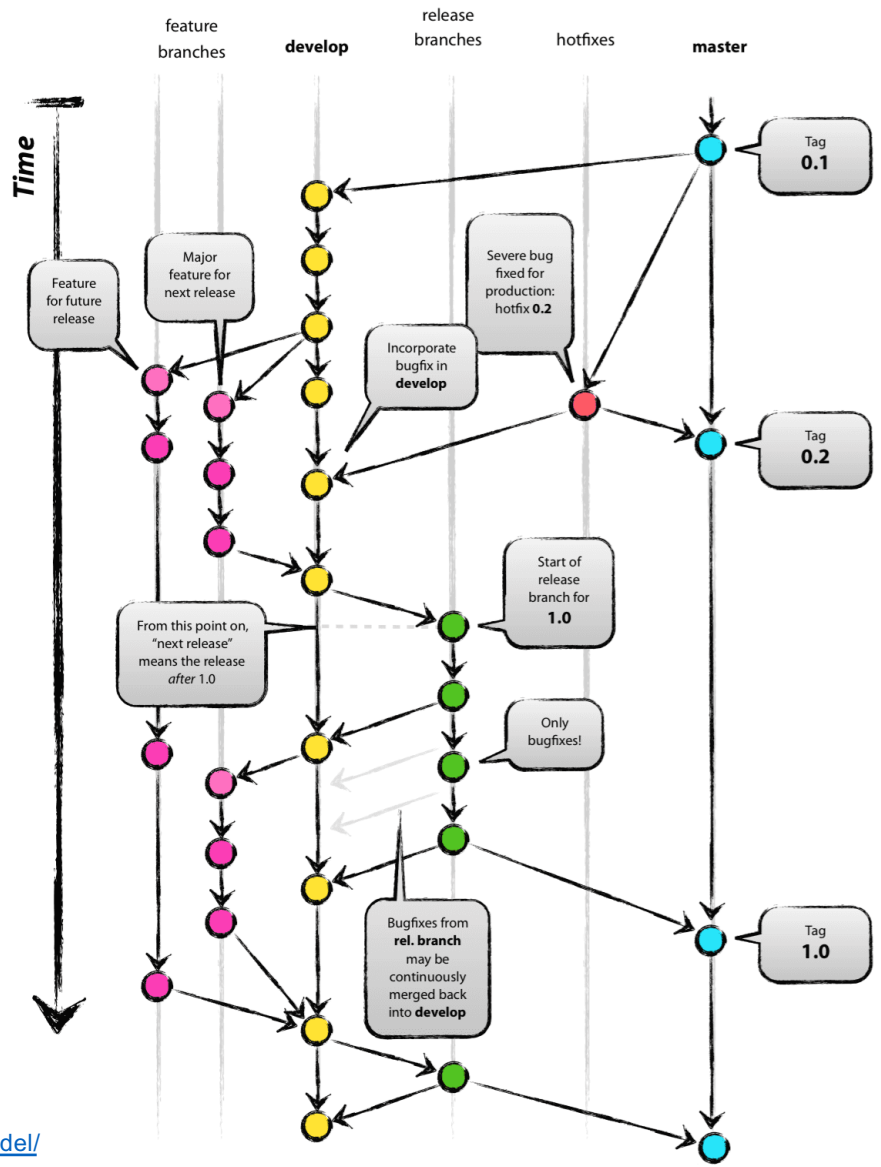
Hotfix Branches

- Hotfix branches arise from the necessity to act immediately upon an undesired state of a live production version.
- When a critical bug in a production version must be resolved immediately, a hotfix branch may be branched off from the corresponding tag on the master branch that marks the production version.



Source: https://nvie.com/posts/a-successful-git-branching-model/

The Whole Picture



Source: https://nvie.com/posts/a-successful-git-branching-model/